

ハッカソン ～計画書と設計書～

チーム 4

25G1062 佐藤 夏美

2026 年 5 月 13 日 Version 0.2

目次

第 1 部	クラリネットパート設計	3
第 1 章	クラリネットパートの基本設計	4
1.1	クラリネットパートの概要	4
1.2	機能一覧	4
1.3	システム構成図	5
1.4	操作インターフェース設計	5
1.5	状態遷移図	6
第 2 章	クラリネットパートの詳細設計	7
2.1	部品一覧	7
2.2	ソフトウェア設計	7
2.2.1	フォーマット	7
2.3	関数仕様	8
2.4	音色パラメータ	8
2.5	Processing 側の処理フロー	10
2.6	テストの設計	11
2.6.1	T1：スペクトル検証	12
2.6.2	T2：聴感比較テスト	12
第 2 部	同期システム設計	13
第 3 章	同期システムの基本設計	14
3.1	同期システム概要	14
3.2	機能一覧	14
3.3	同期システム構成図	15
3.4	操作インターフェース	16
3.4.1	操作者が使用する入力	16
3.4.2	操作者が確認できる出力	16
3.5	状態遷移図（指揮者）	16
第 4 章	同期システムの詳細設計	17
4.1	部品一覧	17
4.2	ハードウェア設計	17
4.2.1	I2C 配線	17
4.2.2	モーター配線	19
4.3	ソフトウェア設計	19

4.3.1	ファイル構成	19
4.4	処理フローの設計	19
4.5	テストの設計	20
4.5.1	実施済みテスト：I2C 同期動作確認	20
4.5.2	今後のテスト	20
4.6	補足：センサーを用いた可変テンポについての思案	20
4.6.1	構成	20
4.6.2	動作の流れ	21
4.6.3	エンターテインメント性	21

第1部 クラリネットパート設計

第 1 章 クラリネットパートの基本設計

本章は、操作者（演奏者・聴き手）の視点からクラリネットパートの構成・動作・状態遷移を定めるものである。プロジェクト全体は 4 楽器（クラリネット・フルート・オルガン・リズム楽器）がそれぞれ独立した Arduino を 1 台ずつ持つ構成をとる。指揮者 Arduino が I2C 経由で各楽器 Arduino に SYNC（同期）メッセージを配信し、各楽器 Arduino はその SYNC メッセージを基準に発音タイミングを制御する。各楽器 Arduino は USB シリアルで音符データを PC へ送信し、PC 上の Processing が各楽器の音色を合成・出力することで合奏が成立する。本章ではそのうちクラリネットパートに関する基本設計および詳細設計を記す。

1.1 クラリネットパートの概要

クラリネット用 Arduino は、「きらきら星」のクラリネットパートを担当する。指揮者用 Arduino の合図に合わせて音符データを PC（Processing）へ送信し、Processing がクラリネットらしい音色で合成・発音する。

1.2 機能一覧

クラリネットパートが提供する機能を以下に示す。

楽曲データ保持機能

クラリネット用 Arduino は、「きらきら星」クラリネットパートの音符列を内部の楽曲配列として事前に保持する。各音符は楽器種別・音程（ノート番号）・音量（ベロシティ）・音長の 4 つで構成される。Arduino は配列の内容を順に送信する。演奏開始までの待機期間は休符として配列に含め、自パートの入りのタイミングから旋律を送信する。

逐次送信機能

演奏開始後、Arduino は指揮者 Arduino からの同期信号を基準に楽曲配列の先頭から 1 音ずつ、設定されたテンポに基づく発音タイミングが来るたびに次の音符データを USB シリアル経由で PC へ送信する。送信フォーマットはチーム共通の 5 バイトバイナリ形式とし、ヘッダ、楽器種別、音程、音量、音長を持つ。テンポは SYNC メッセージに含まれる BPM 値を受信することで更新され、可変に対応する。

クラリネット音色合成・再生機能

Processing が Arduino からデータを受信次第、音程（ノート番号）・音量（ベロシティ）・音長などから適切な音色を即時発音する。音色は Minim ライブラリによる加算合成にローパス

フィルタおよび ADSR エンベロープを組み合わせた構成で、実測 FFT スペクトルから算出した倍音テーブルを用いて閉管楽器特有の奇数倍音優勢のスペクトルを再現する。

1.3 システム構成図

クラリネットパートに関わる構成要素を図 1.1 に示す。

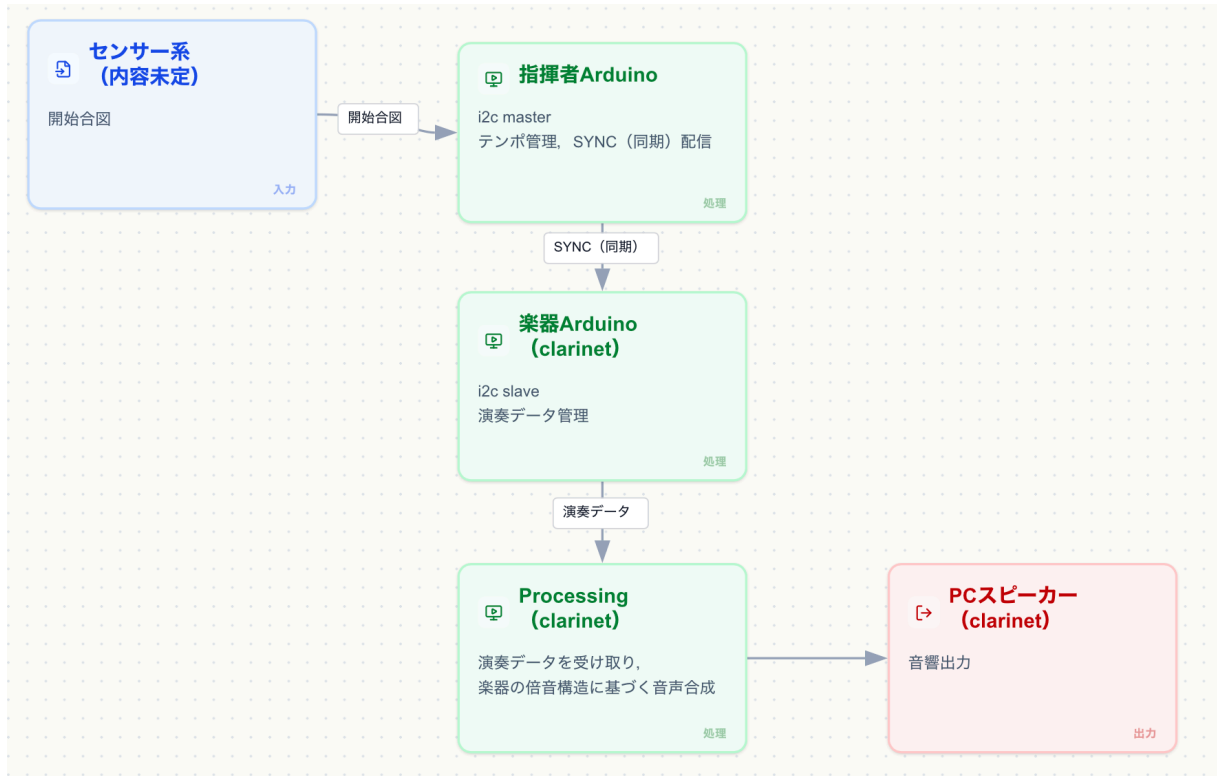


図 1.1 クラリネットパートのシステム構成（フルート・オルガン・リズム楽器も同様）

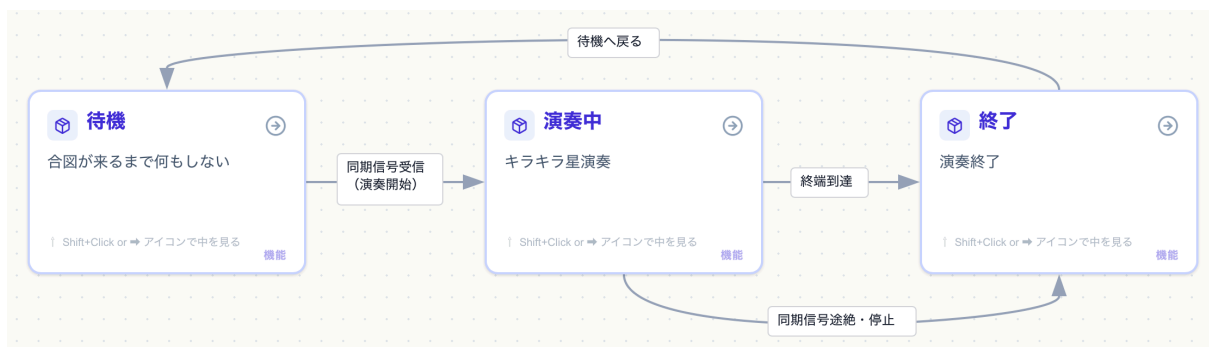
Arduino が楽曲データの保持と逐次送信を担当し、PC 上の Processing が受信・音色合成・音響出力を担当する。音声出力はすべて Processing（PC 側）が行う。

1.4 操作インターフェース設計

演奏の開始は全楽器一斉に行うため、クラリネット単体での開始スイッチは設けない。全体の演奏制御（開始の合図・テンポ変更）は指揮者 Arduino 側で行い、クラリネット用 Arduino はその同期信号を受けて動作する。

1.5 状態遷移図

クラリネットパートの状態遷移を図 1.2 に示す。状態は「待機」「演奏中」「終了」の3つとし、指揮者 Arduino からの開始合図の受信および楽曲終端への到達で遷移が起こる。



演奏中は入りまでの期間を休符として管理し、SYNC メッセージで受け取った BPM をもとに Arduino が発音タイミングを計算して音符データを送信する。Processing 側は音符データを受信次第即時発音する。

第2章 クラリネットパートの詳細設計

本章では、クラリネットパートを実装するための詳細設計を記す。

2.1 部品一覧

表 2.1 クラリネットパートの部品一覧

分類	部品名	用途
ハードウェア	Arduino UNO R4 Wi-Fi	楽曲配列保持・逐次送信
ハードウェア	USB ケーブル	PC 接続・電源供給
ハードウェア	PC (スピーカー内蔵)	Processing 動作・音響出力
ソフトウェア	Arduino IDE	楽曲データ等書き込み
ソフトウェア	Processing	音色合成・再生
ライブラリ	Minim (ddf.minim.*)	Oscil, MoogFilter, ADSR, Noise, WavetableGenerator

2.2 ソフトウェア設計

2.2.1 フォーマット

クラリネット用 Arduino から PC への送信はチーム共通の 5 バイトバイナリ形式で行う。バイナリ形式とはコンピュータが理解できる「0」と「1」の 2 進数（ビット）で情報を表現するデータ形式である。バイナリ形式を採用した理由は、テキスト送信と比べてデータ量が少なく通信が軽量になるためである。またノート番号についても、きらきら星で使用するドレミファソラの 6 音に対して独自の番号（0～5）を割り当てることで 1 バイトに収める。

注意：ヘッダバイトの値および楽器種別の番号割り当ては 4 楽器合わせてチームで統一する必要がある（現時点では仮値）。

表 2.2 1 音あたりの送信フレーム (5 バイト, 仮値)

バイト	名称	値 (仮)	意味
1 バイト目	ヘッダ	0xAA	フレーム開始の目印 (同期バイト)
2 バイト目	楽器種別	0-3	フルート=0, クラリネット=1, オルガン=2, リズム=3
3 バイト目	ノート番号	0-5	きらきら星で使う音の番号. 0=C4, 1=D4, 2=E4, 3=F4, 4=G4, 5=A4
4 バイト目	ベロシティ	0-127	音の強さ. Processing 側で 0.0~1.0 に正規化し振幅に反映する.
5 バイト目	音長	0-255	10 ミリ秒単位. Processing 側で 10 を掛けてミリ秒に戻す. 音長の終端で Processing が自動的に noteOff() を呼び消音する.

2.3 関数仕様

ClarinetInstrument(float freq, float velocity, float length)

引数 freq: 音の周波数 (Hz), velocity: 振幅 (0.0~1.0, ベロシティを正規化した値), length: 音長 (秒)

戻り値 なし (コンストラクタ)

内容 音声合成の処理経路を構築する. トーン系統, リードノイズ系統, 低域空気感系統を生成する.

noteOn(float duration)

引数 duration: 発音時間 (秒)

戻り値 なし

内容 Minim の playNote() から自動的に呼ばれる. mainEnv・reedEnv・breathEnv を AudioOutput にパッチして発音を開始する. duration 秒後に noteOff() が自動的に呼ばれる.

noteOff()

引数 なし

戻り値 なし

内容 Minim から自動的に呼ばれる. mainEnv・reedEnv・breathEnv それぞれの noteOff() を呼び, Release フェーズ終了後に unpatchAfterRelease(out) で自動切断する.

2.4 音色パラメータ

表 2.3 クラリネット音色のパラメータ

パラメータ	値
ベース波形	加算合成 (WavetableGenerator.gen10)
倍音構成	h1~h16 (奇数倍音優勢)
トーン用 LPF 遮断周波数	音程連動 (ノート周波数に比例)
mainEnv (トーン用 ADSR)	
Attack	0.400 s (仮)
Decay	0.040 s (仮)
Sustain	0.95 (仮)
Release	0.300 s (仮)
reedEnv (リードノイズ用 ADSR)	
帯域	1200 Hz~2800 Hz (仮)
Attack	0.010 s (仮)
Sustain	0.05 (仮)
Release	0.080 s (仮)
breathEnv (低域空気感用 ADSR)	
帯域	80 Hz 以下 (仮)
Attack・Release	mainEnv と同じ (仮)
全体振幅	ベロシティに連動

各値の根拠を以下に記す。

ベース波形：矩形波は倍音振幅が $1/h$ の固定比率のため、クラリネットの細かい特徴を再現できない。加算合成を採用し、表 2.4 の実測 FFT 解析値をもとに倍音振幅を設定した。

表 2.4 実測 FFT 解析結果 (クラリネット C4, Audacity・Hann 窓)

倍音	種別	実測周波数	実測レベル [dB]	基音比 [dB]
第 1 (基音)	奇数	263 Hz (C4)	-23.2	0
第 2	偶数	523 Hz (C5)	-71.0	-47.8
第 3	奇数	788 Hz (G5)	-21.5	+1.7
第 4	偶数	1049 Hz (C6)	-56.0	-32.8
第 5	奇数	1312 Hz (E6)	-30.2	-7.0
第 6	偶数	1573 Hz (G6)	-62.4	-39.2
第 7	奇数	1835 Hz (Ab6)	-40.8	-17.6
第 8	偶数	2097 Hz (C7)	-51.4	-28.2

ADSR：トーン・リードノイズ・低域空気感の 3 系統を独立して制御することで、リードが振動し始めるクラリネット固有の発音特性を再現する方針とする。

フィルタ：カットオフをノート周波数に連動させることで、音程によらず一定の音色を保つ方針とする。

2.5 Processing 側の処理フロー

クラリネットパートの処理フローを図 2.1 に示す。

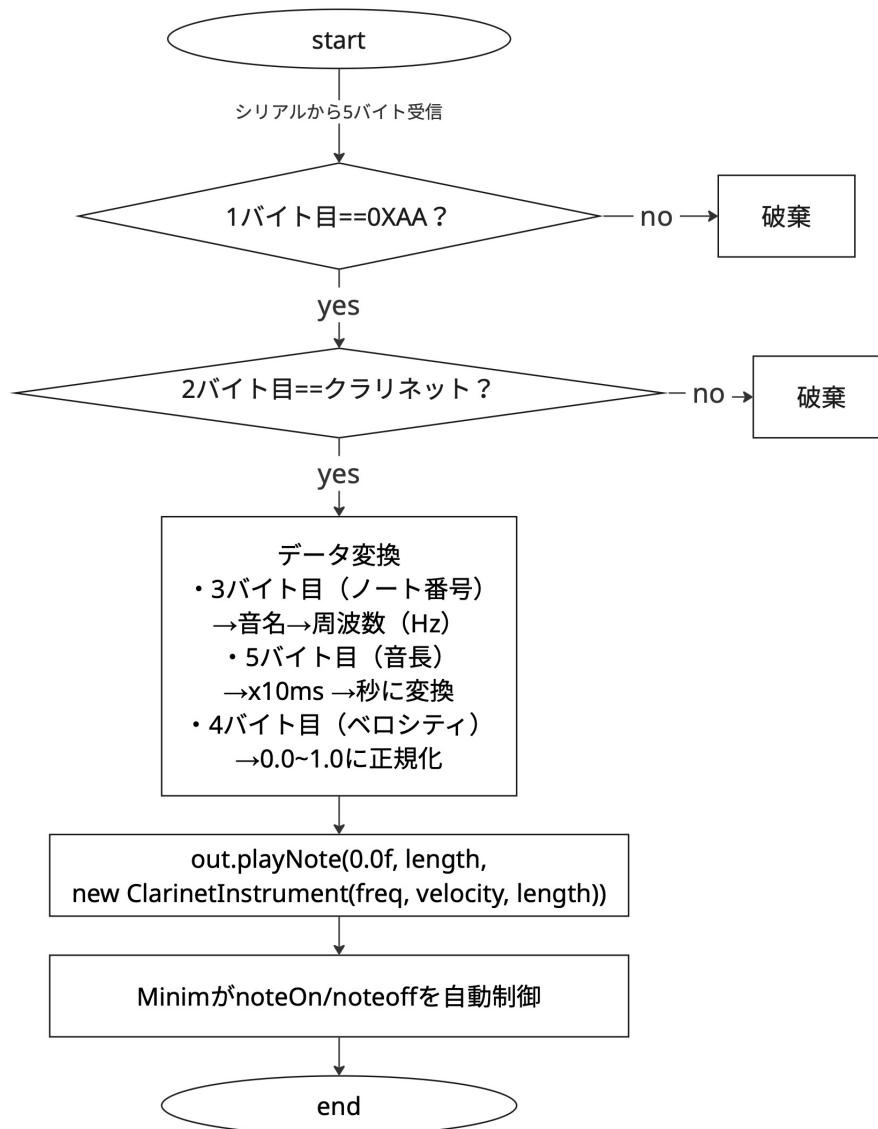


図 2.1 クラリネットパートの処理フロー

2.6 テストの設計

2.6.1 T1：スペクトル検証

Audacity で FFT 解析を行い，実測クラリネット音源（freesound.org 番号 248359，C4 相当）と比較する．奇数倍音中心の倍音構造などを確認する．

Audacity の Hann 窓に設定してスペクトル解析を行い，表 2.4 の実測値と比較する．FFT サイズが小さい場合（例：4096）は周波数分解能が低くピークが横に広がって見えるため，同一条件での比較が必要である．

2.6.2 T2：聴感比較テスト

実際のクラリネット単音録音と合成音を並べて再生し，「クラリネットらしく聞こえるか」をチームメンバーで聴認する．管楽器らしい立ち上がりなどを確認する．

第2部 同期システム設計

第 3 章 同期システムの基本設計

本章は、操作者（演奏者）の視点から同期システムの構成・操作方法・状態遷移を定めるものである。プロジェクト全体は 4 楽器（クラリネット・フルート・オルガン・リズム楽器）がそれぞれ独立した Arduino を 1 台ずつ持つ構成をとる。指揮者 Arduino が I2C 経由で各楽器 Arduino に SYNC メッセージを配信し、各楽器 Arduino はその SYNC メッセージを基準に発音タイミングを制御する。各楽器 Arduino は USB シリアルで音符データを PC へ送信し、PC 上の Processing が各楽器の音色を合成・出力することで合奏が成立する。本章ではそのうち同期システムに関する基本設計および詳細設計を記す。

3.1 同期システム概要

複数の Arduino が指揮者 Arduino の合図に合わせて「きらきら星」を輪唱形式で演奏するシステムである。モーターが 1 周したことを検知すると演奏が開始される。各楽器は自身の入りまでの期間を休符として管理し、楽器 A・B・C が 2 小節ずつずれて順番に旋律の演奏を開始することで、時間差で同一の旋律が重なり合う輪唱が成立する。

曲目 きらきら星（12 小節，4/4 拍子）

編成 指揮者兼モーター制御 Arduino × 1，楽器 Arduino × 3，リズム Arduino × 1（計 5 台）

操作者の入力 モーターの回転による演奏開始の合図

システムの出力 楽器音（スピーカー）

3.2 機能一覧

演奏開始検知機能

指揮者 Arduino はモーターの 1 周を検知し、演奏開始の合図とする。検知方法の詳細は未確定（相談中）。

SYNC メッセージ配信機能

指揮者 Arduino は毎小節頭に現在の小節番号（`global_bar`）と BPM を I2C 経由で全スレーブへ順番に送信する。

PLL 補正機能

各楽器 Arduino（スレーブ）は SYNC メッセージを受信するたびに、予測時刻と実際の受信時刻の差を計算し次の小節長を微調整する。これによりスレーブ個体のクリスタルドリフト（水晶発振子の個体差による時刻誤差の蓄積）を防ぎ、全楽器が指揮者 Arduino に追従する。

輪唱機能（各スレーブにて設定するものである。）

各楽器 Arduino は入りまでの期間を休符として管理し、所定の小節から旋律の演奏を開始する。具体的には、以下のタイミングで各パートが順次発音を開始する。

- 0 小節目 → 楽器 A・リズムが演奏開始（楽器 B・C は休符）
- 2 小節目 → 楽器 B が演奏開始（楽器 C は休符）
- 4 小節目 → 楽器 C が演奏開始

全楽器が同一の指揮者 Arduino に追従することで、楽器間の直接通信なしに同期が実現される。

3.3 同期システム構成図

同期システムに関わる構成要素を図 3.1 に示す。ただし、全体の流れがわかるように各スレーブ以降の流れも示している。

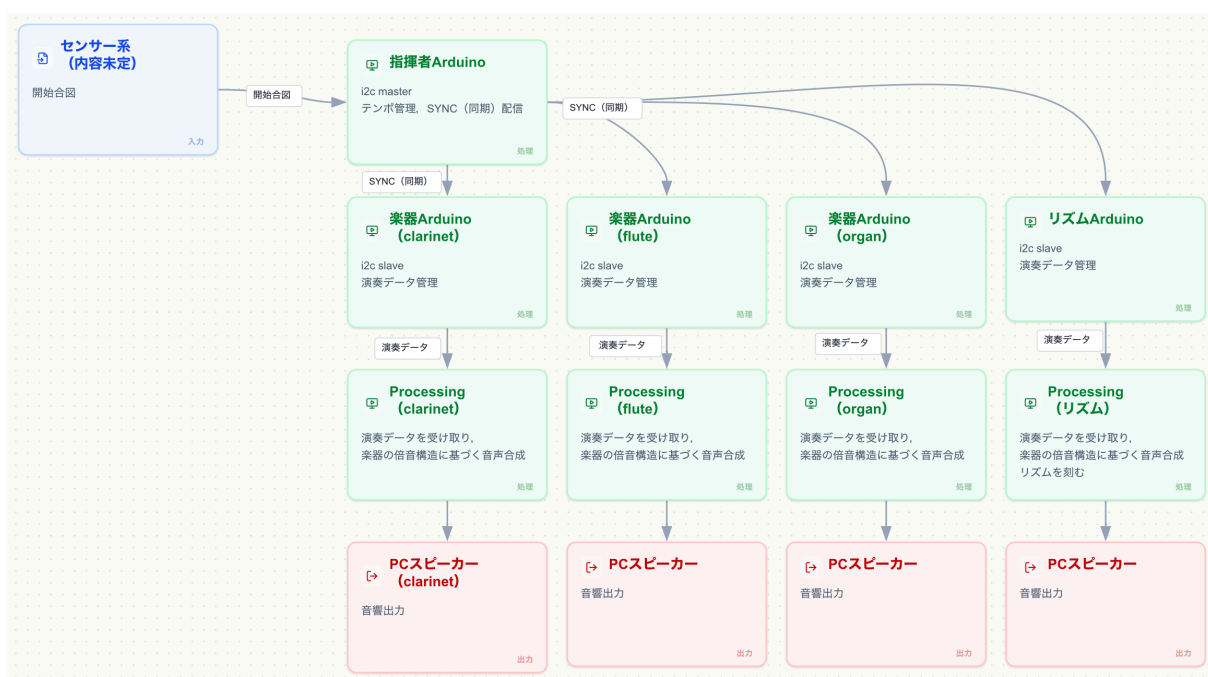


図 3.1 同期システム全体構成図

指揮者 Arduino がモーターの 1 周を検知して他の全 Arduino へ開始合図（SYNC）を送信すると、合図を受けた各 Arduino（楽器・リズム）が自身の担当パートの演奏データを PC へ送り、最終的に PC 上の Processing がそのデータをもとに音色を合成してスピーカーから出力する。

3.4 操作インターフェース

3.4.1 操作者が使用する入力

表 3.1 操作者の入力一覧

入力	場所	動作
電源 ON	指揮者 Arduino	システム起動
電源 OFF	指揮者 Arduino	演奏停止

3.4.2 操作者が確認できる出力

表 3.2 操作者への出力一覧

出力	場所	内容
音声	スピーカー（PC）	各楽器の音色で輪唱が鳴る
モーターの回転	指揮者 Arduino	1 周検知で演奏開始

3.5 状態遷移図（指揮者）

同期システムの状態遷移を図 3.2 に示す。状態は「電源オフ」「待機」「演奏中」「演奏終了」の 4 つとし、電源投入により待機状態となる。その後、モーターの 1 周検知を演奏開始の合図として「演奏中」へ遷移し、同期データの送信を行う。楽曲の終端へ到達すると「演奏終了」へ遷移し、その後は次の演奏に向けて「待機」状態へ戻るか、電源オフとなる。



図 3.2 状態遷移図（指揮者）

第 4 章 同期システムの詳細設計

本章では，同期システムを実装するための詳細設計を記す．

4.1 部品一覧

表 4.1 同期システムの部品一覧

分類	部品名	用途
ハードウェア	Arduino UNO R4 Wi-Fi × 5	指揮者兼モーター制御 1 台・楽器 3 台・リズム 1 台
ハードウェア	PC (スピーカー) × 5	Processing での音色合成・音響出力
ハードウェア	ジャンパ線	I2C 配線 (SDA・SCL・GND)
ハードウェア	モーター	モーター 1 周の回転駆動
ハードウェア	USB ケーブル × 5	PC 接続・電源供給
ソフトウェア	Arduino IDE	プログラム書き込み
ライブラリ	Wire.h	I2C 通信
ライブラリ	Arduino_LED_Matrix.h	LED マトリクス表示

4.2 ハードウェア設計

4.2.1 I2C 配線

試作した同期システムの配線を図 4.1，図 4.2 に示す．I2C はバス接続のため，全 Arduino の SDA・SCL・GND を共有する．実際はスレーブがあと 2 つ接続される．

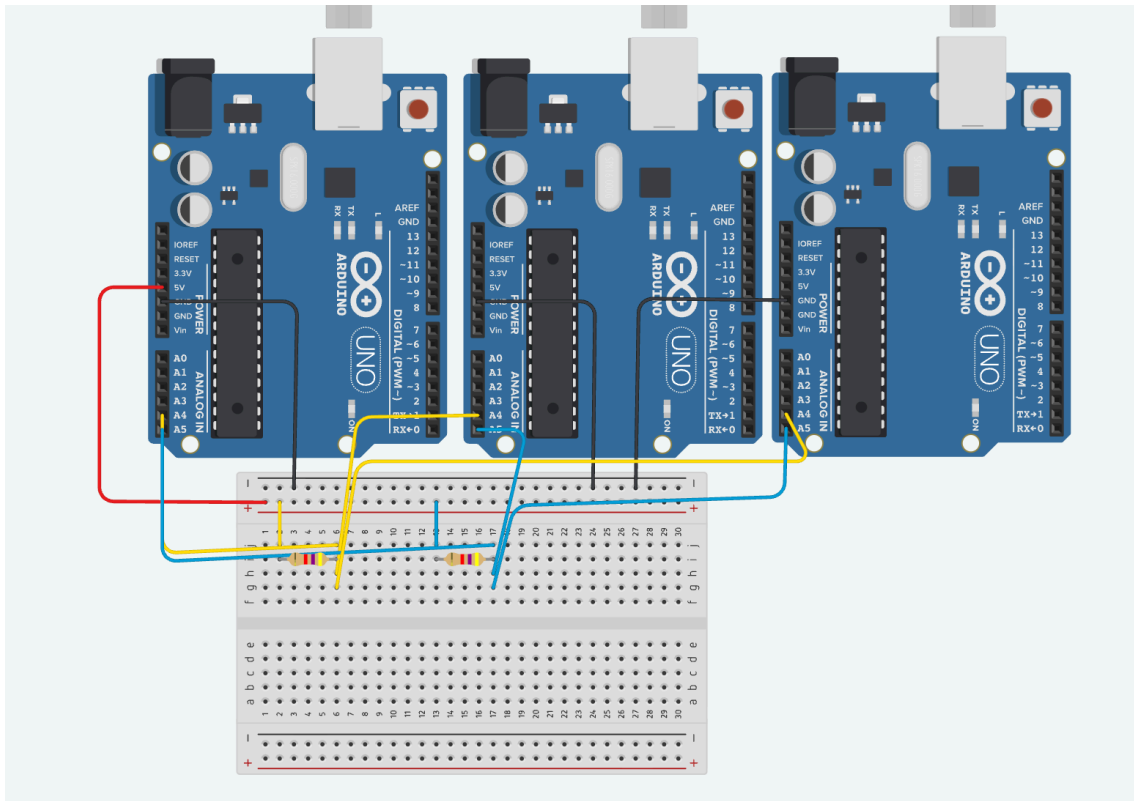


図 4.1 I2C 配線

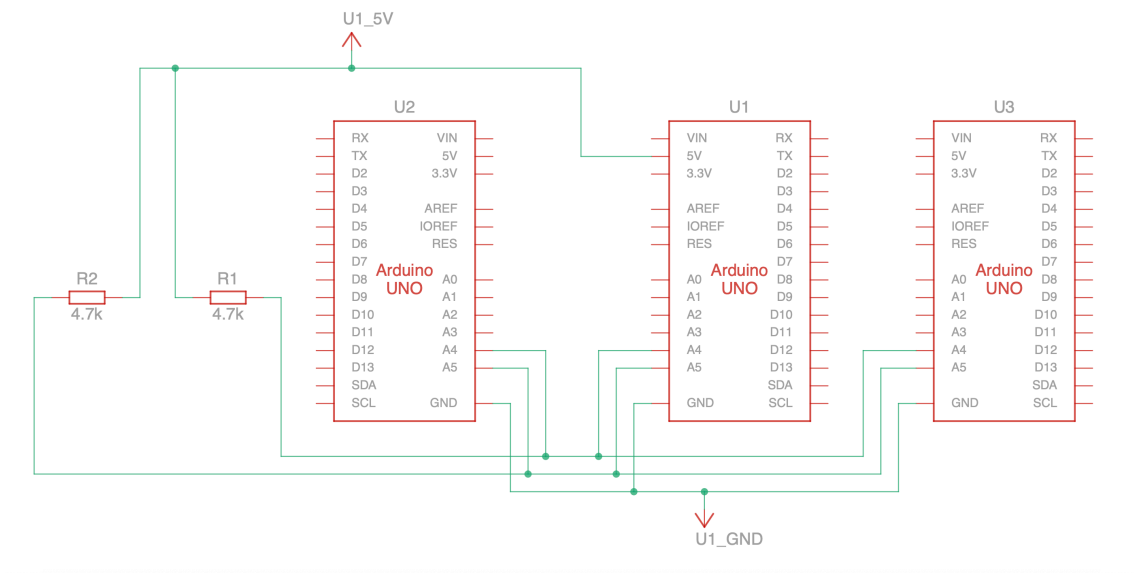


図 4.2 I2C 配線 (回路図)

注意：Arduino の Wire ライブラリはスレーブに 1 台ずつ順番に送信するため，スレーブごとに受信タイミングが約 0.5 ms ずつ異なる．このズレは蓄積しない固定オフセットであり，人間の聴覚閾値（約 20～30 ms）を大幅に下回るため，現時点では許容している．

4.2.2 モーター配線

モーターの具体的な配線・ピン割り当ては未確定（実機テストで確定予定）．

4.3 ソフトウェア設計

4.3.1 ファイル構成

表 4.2 ファイル構成

ファイル名	内容
master.ino	指揮者 Arduino：SYNC メッセージ配信・テンポ管理
slave1.ino	楽器 Arduino：SYNC メッセージ受信・発音タイミング制御
slave2.ino	楽器 Arduino：SYNC メッセージ受信・発音タイミング制御
slave3.ino	楽器 Arduino：SYNC メッセージ受信・発音タイミング制御
slave4.ino	楽器 Arduino：SYNC メッセージ受信・発音タイミング制御

現時点では簡単な同期動作確認として master.ino・slave1.ino・slave2.ino の 3 ファイルで動作の確認を行った．slave1.ino と slave2.ino は I2C アドレスのみ異なり（0x10 と 0x11），それ以外の処理は同一である．

4.4 処理フローの設計

同期システムの処理フローを図 4.3 に示す．

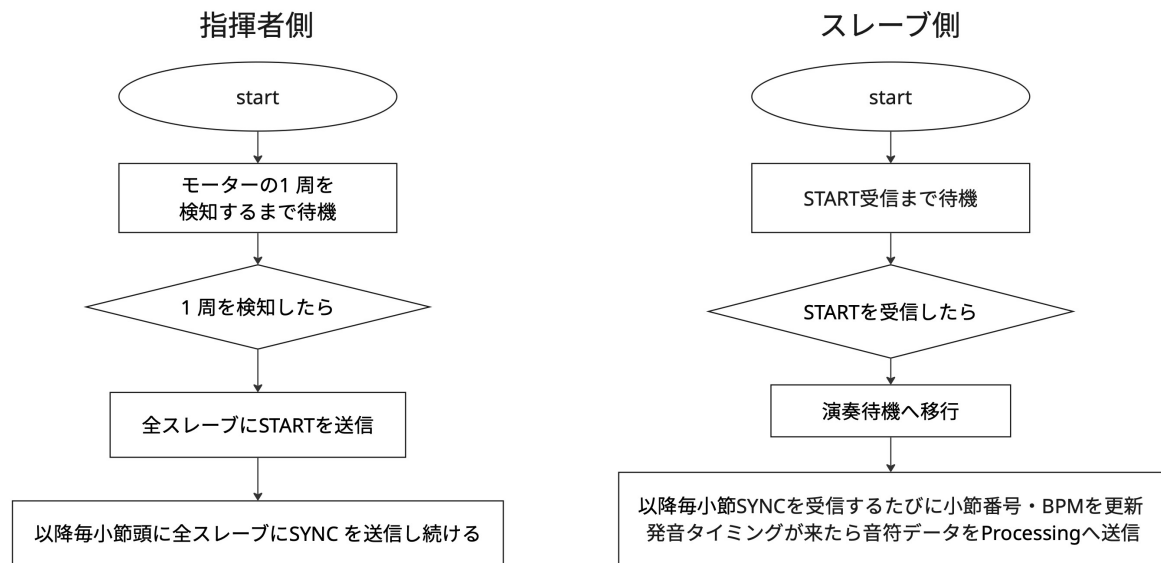


図 4.3 指揮者とスレーブについての同期システムの処理フロー

4.5 テストの設計

4.5.1 実施済みテスト：I2C 同期動作確認

指揮者 Arduino 1 台とスレーブ Arduino 2 台を実機接続し、I2C 経由で CONFIG・START・SYNC の送受信が正しく動作することを確認した。

4.5.2 今後のテスト

- モーターの 1 周検知による演奏開始の動作確認
- PLL 補正を実装した後の長時間同期精度の確認
- 楽器 Arduino との組み合わせによる輪唱動作の確認

4.6 補足：センサーを用いた可変テンポについての思案

現時点では固定 BPM での同期動作確認にとどまっているが、エンターテインメント性の向上を目的として以下の可変テンポ機能の実装を思案している。

4.6.1 構成

モーター ステッピングモーター（28BYJ-48 5V）+ ULN2003 ドライバモジュール

速度調整 可変抵抗（つまみ）によるモーターの回転速度の調整

1 周検知 フォトリフレクタによるモーターの 1 周検知

4.6.2 動作の流れ

操作者がつまみを操作することでモーターの回転速度が変化する。フォトリフレクタが 1 周を検知するたびに 1 周にかかった時間を計測し、時間の範囲に応じて BPM を段階的に分類する（表 4.3 では例として 3 段階に設定）。分類された BPM は次の小節頭の SYNC メッセージに乗せて全楽器へ反映される。

表 4.3 1 周の時間と BPM の対応（具体的な値は実機テストで確定）

1 周にかかった時間	BPM
短い（高速回転）	速い（例：120 BPM）
中程度	普通（例：90 BPM）
長い（低速回転）	遅い（例：60 BPM）

具体的な BPM の値と時間の範囲は 28BYJ-48 の実際の回転速度を実機で計測した上で確定する予定である。

4.6.3 エンターテインメント性

モーターの回転速度がそのまま音楽のテンポに対応するため、操作者がモーターの回る速さを変えると音楽のテンポも変わるという視覚と音楽が連動した演出が実現できる。